CT-2 Report

Title: Decision Tree Classification Using Student Performance Dataset

Course title: ***Data Mining Problem Description***

Course code: CSE-452
4th Year 1st Semester Examination 2025

Date of Submission: 14/05/2026

Submitted to-

Md. Humayun Kabir
Professor
Department of Computer Science and Engineering
Jahangirnagar University

&

Dr. Md Musfique Anwar
Professor
Department of Computer Science and Engineering
Jahangirnagar University
Savar, Dhaka-1342

Class Roll	Exam Roll	Name
383	210903	Abdullah Nazmus-Sakib

1. Experiment Name

Design and Implementation of Decision Tree Classification Model Using Student Performance Dataset

---

2. Objective

The objective of this experiment is to design and implement a **Decision Tree Prediction Model** for classification using student academic performance data.

The model is trained using GPA-related features and predicts the performance category of students such as:

- Excellent
- Good
- Average
- Poor

The experiment also aims to analyze the classification accuracy and visualize the generated decision tree.

---

3. Algorithm for Decision Tree Construction

Decision Tree Algorithm Hunt (J48 / C4.5 for formating)

The Decision Tree algorithm works by recursively splitting the dataset based on the most important attribute.

Steps of the Algorithm

1. Load the dataset from CSV file.
2. Select the best attribute for splitting.
3. Divide the dataset into subsets.
4. Create decision nodes and branches.
5. Continue recursively until all records belong to the same class.
6. Generate leaf nodes for final classes.

Attribute Used

- FirstYear GPA
- SecondYear GPA

- ThirdYear GPA
- AVG_Till_Now GPA

Target Class

- Excellent
- Good
- Average
- Poor

---

4. Pseudocode

Procedure: connectCSV()

```
procedure connectCSV(file_path)
1: open CSV file
2: read dataset
3: return dataset
```

Procedure: createNode()

```
procedure createNode(attribute, threshold)
1: create new node
2: assign attribute and threshold
3: return node
```

Procedure: split()

```
procedure split(dataset, feature, threshold)
1: divide dataset into left and right
2: left = feature <= threshold
3: right = feature > threshold
4: return left, right
```

Procedure: buildTree()

```
procedure buildTree(dataset)
1: if all data belong to same class
2: return leaf node
3: find best split
4: split dataset
5: recursively build left subtree
6: recursively build right subtree
7: return decision tree
```

---

5. Data Sample and Connection

Sample Dataset

FirstYear	SecondYear	ThirdYear	AVG_Till_Now	Performance_Class
3.875	4.00	1.975	3.94	Excellent
3.32	3.87	1.95	3.656	Good
3.14	3.49	1.755	3.354	Average
2.855	3.14	1.68	3.07	Poor

Total Instances: **52**

CSV File Connection

```
import pandas as pd

dataset = pd.read_csv("student_performance.csv")
print(dataset.head())
```

---

6. Python Code

Individual code

```
class Node:
    def __init__(self, feature=None, threshold=None, left=None, right=None,
value=None):
        self.feature = feature
        self.threshold = threshold
        self.left = left
        self.right = right
        self.value = value

def split(dataset, feature, threshold):
    left = dataset[dataset[feature] <= threshold]
    right = dataset[dataset[feature] > threshold]
    return left, right

def build_tree(dataset):
    if len(set(dataset['label'])) == 1:
        return dataset['label'].iloc[0]

    feature = 'AVG_Till_Now'
    threshold = dataset[feature].mean()
```

```
    left, right = split(dataset, feature, threshold)

    return Node(
        feature=feature,
        threshold=threshold,
        left=build_tree(left),
        right=build_tree(right)
    )
```

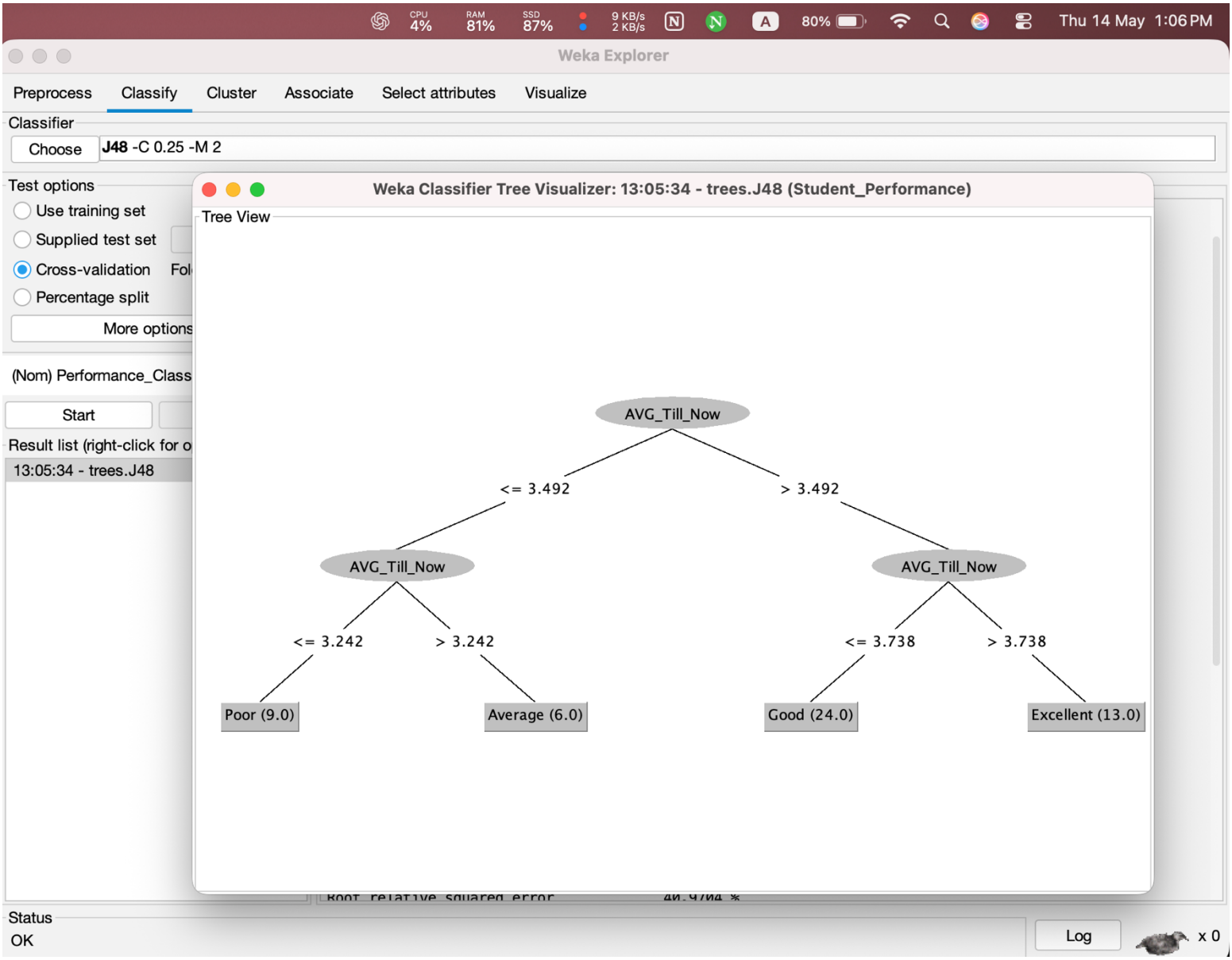
Sample complete code code

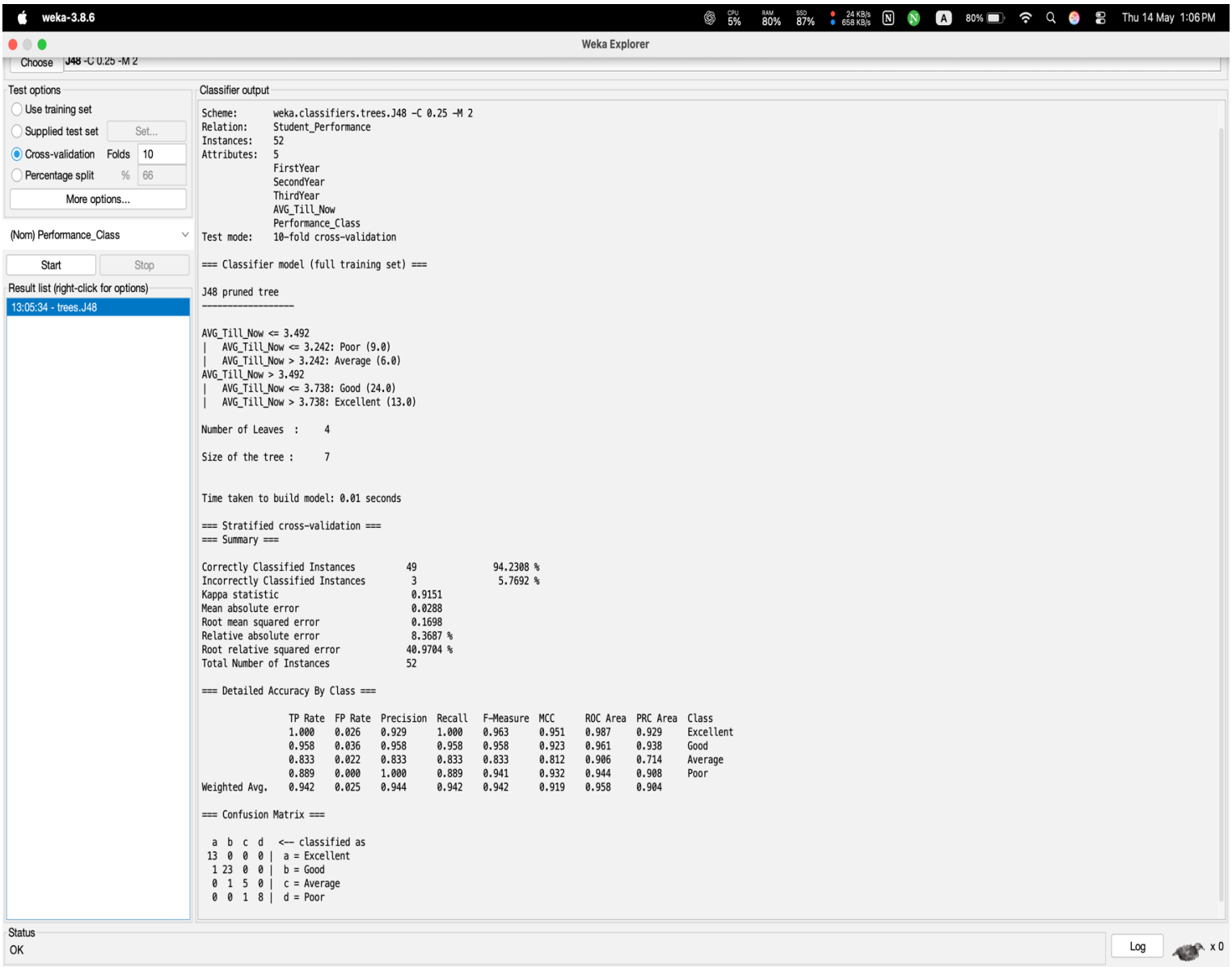
```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree
import matplotlib.pyplot as plt

data = pd.read_csv("student_performance.csv")

X = data[["FirstYear", "SecondYear", "ThirdYear", "AVG_Till_Now"]]

y = data["Performance_Class"]

model = DecisionTreeClassifier()

model.fit(X, y)

prediction = model.predict([[3.7, 3.8, 1.9, 3.75]])

print("Predicted Class:", prediction)

plt.figure(figsize=(12,8))
tree.plot_tree(
    model,
    feature_names=X.columns,
    class_names=model.classes_,
    filled=True
)

plt.show()
```

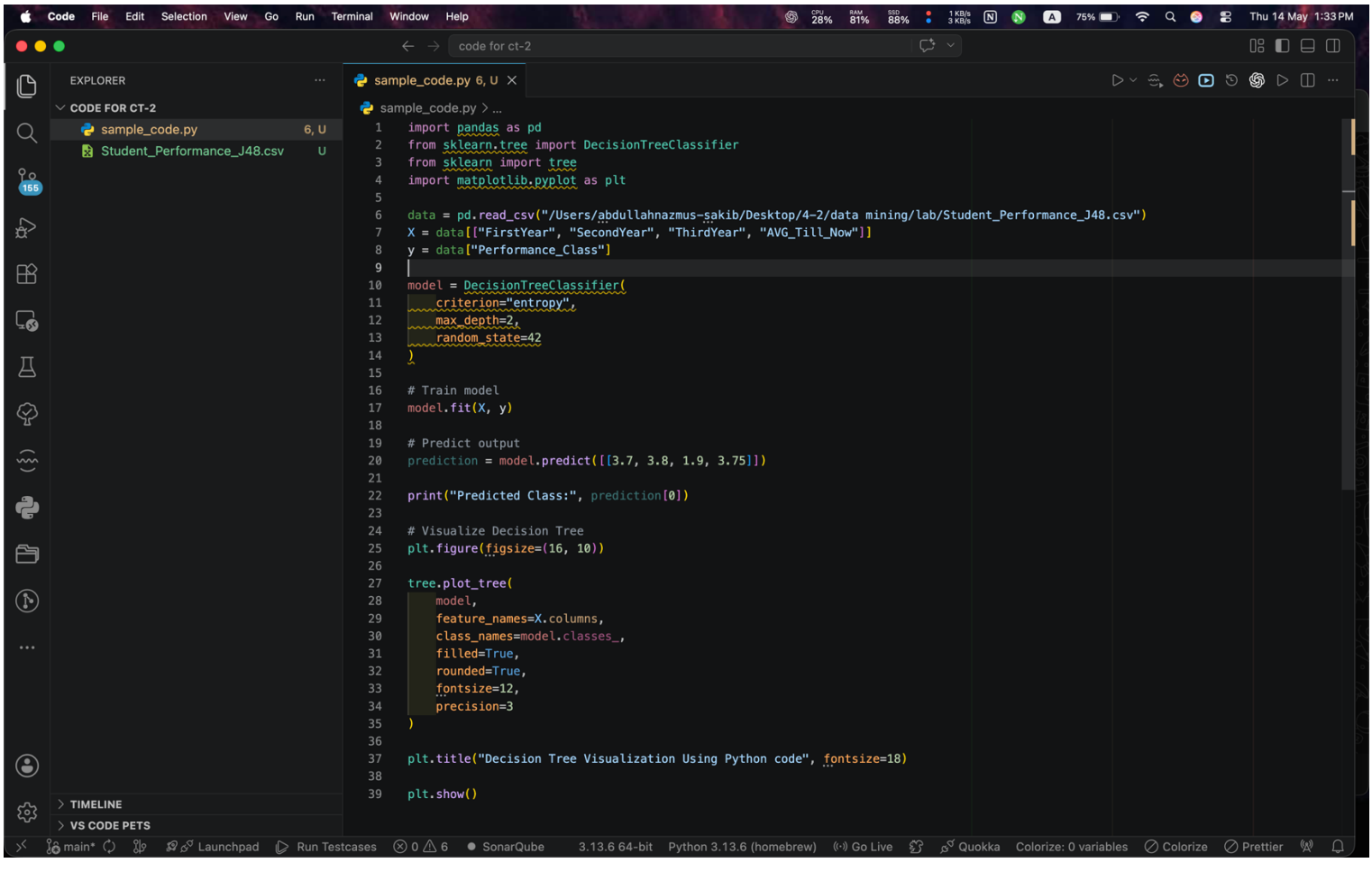
---

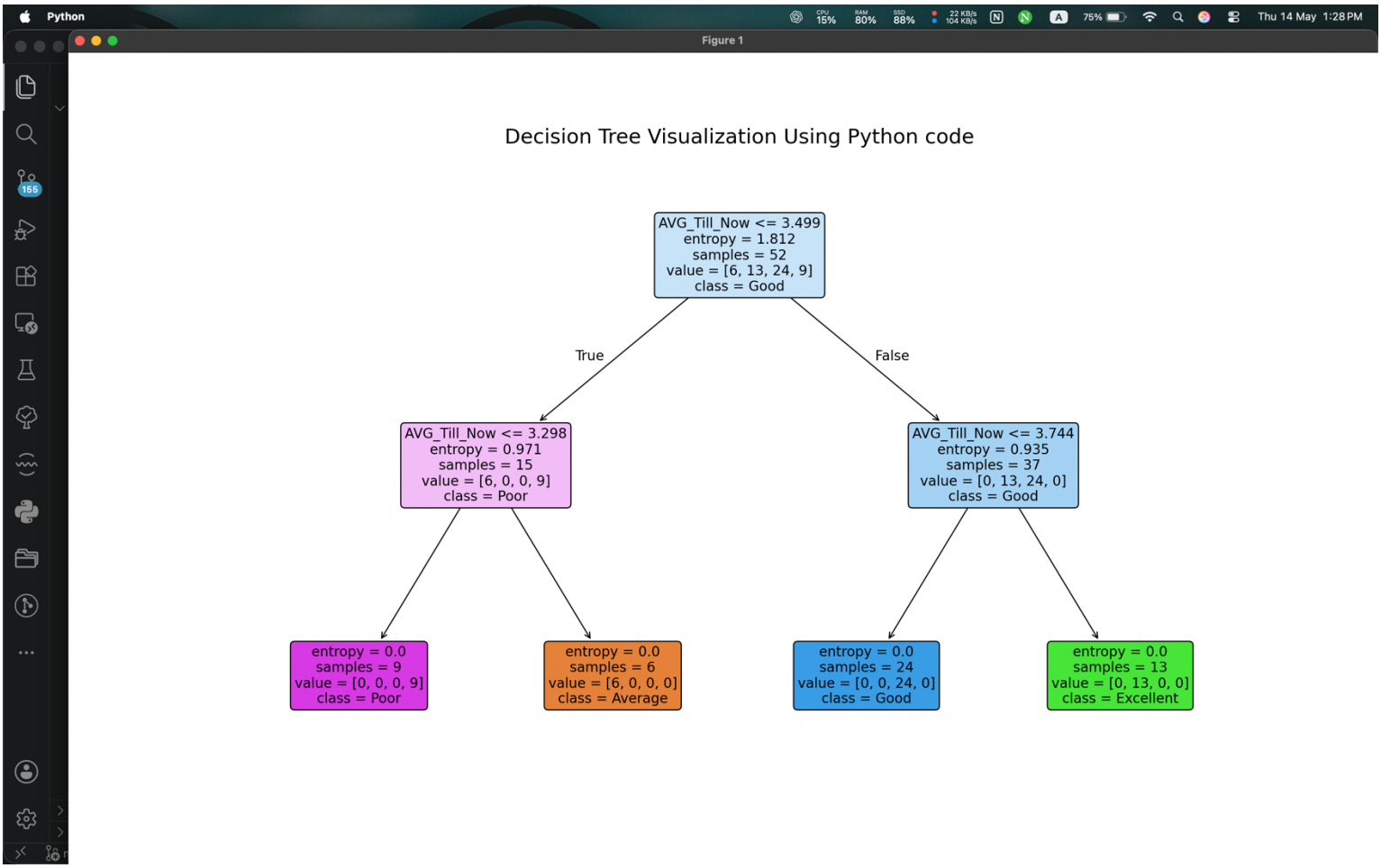
7. Result and Discussion

Obtained Decision Tree




weka-3.8.6
Thu 14 May 1:06 PM
Weka Explorer
Choose J48 -C 0.25 -M 2
Test options
Use training set
Supplied test set Set...
Cross-validation Folds 10
Percentage split % 66
More options...
(Nom) Performance_Class
Start Stop
Result list (right-click for options)
13:05:34 - trees.J48
Classifier output
Scheme: weka.classifiers.trees.J48 -C 0.25 -M 2
Relation: Student_Performance
Instances: 52
Attributes: 5
FirstYear
SecondYear
ThirdYear
AVG_Till_Now
Performance_Class
Test mode: 10-fold cross-validation
=== Classifier model (full training set) ===
J48 pruned tree
AVG_Till_Now <= 3.492
| AVG_Till_Now <= 3.242: Poor (9.0)
| AVG_Till_Now > 3.242: Average (6.0)
AVG_Till_Now > 3.492
| AVG_Till_Now <= 3.738: Good (24.0)
| AVG_Till_Now > 3.738: Excellent (13.0)
Number of Leaves : 4
Size of the tree : 7
Time taken to build model: 0.01 seconds
=== Stratified cross-validation ===
=== Summary ===
Correctly Classified Instances 49 94.2308 %
Incorrectly Classified Instances 3 5.7692 %
Kappa statistic 0.9151
Mean absolute error 0.0288
Root mean squared error 0.1698
Relative absolute error 8.3687 %
Root relative squared error 40.9704 %
Total Number of Instances 52
=== Detailed Accuracy By Class ===
TP Rate FP Rate Precision Recall F-Measure MCC ROC Area PRC Area Class
1.000 0.026 0.929 1.000 0.963 0.951 0.987 0.929 Excellent
0.958 0.036 0.958 0.958 0.958 0.923 0.961 0.938 Good
0.833 0.022 0.833 0.833 0.833 0.812 0.906 0.714 Average
0.889 0.000 1.000 0.889 0.941 0.932 0.944 0.908 Poor
Weighted Avg. 0.942 0.025 0.944 0.942 0.942 0.919 0.958 0.904
=== Confusion Matrix ===
a b c d <-- classified as
13 0 0 0 | a = Excellent
1 23 0 0 | b = Good
0 1 5 0 | c = Average
0 0 1 8 | d = Poor
Status
OK
Log
x 0


Code File Edit Selection View Go Run Terminal Window Help
Thu 14 May 1:33 PM
code for ct-2
EXPLORER
CODE FOR CT-2
sample_code.py 6, U
Student_Performance_J48.csv U
sample_code.py 6, U
sample_code.py > ...
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree
import matplotlib.pyplot as plt
data = pd.read_csv("/Users/abdullahnazmus-sakib/Desktop/4-2/data mining/lab/Student_Performance_J48.csv")
X = data[["FirstYear", "SecondYear", "ThirdYear", "AVG_Till_Now"]]
y = data["Performance_Class"]
model = DecisionTreeClassifier(
criterion="entropy",
max_depth=2,
random_state=42
)
# Train model
model.fit(X, y)
# Predict output
prediction = model.predict([[3.7, 3.8, 1.9, 3.75]])
print("Predicted Class:", prediction[0])
# Visualize Decision Tree
plt.figure(figsize=(16, 10))
tree.plot_tree(
model,
feature_names=X.columns,
class_names=model.classes_,
filled=True,
rounded=True,
fontsize=12,
precision=3
)
plt.title("Decision Tree Visualization Using Python code", fontsize=18)
plt.show()
TIMELINE
VS CODE PETS
main* Launchpad Run Testcases 0 6 SonarQube 3.13.6 64-bit Python 3.13.6 (homebrew) Go Live Quokka Colorize: 0 variables Colorize Prettier



Result Analysis

The Decision Tree model successfully classified student performance based on GPA values.

Performance Summary

- Correctly Classified Instances: 49
- Incorrectly Classified Instances: 3
- Accuracy: 94.23%
- Number of Leaves: 4
- Tree Size: 7

The experiment shows that **AVG_Till_Now** is the most important attribute for classification.

Students with higher GPA are classified as **Excellent**, while lower GPA students are classified as **Poor**.

The model achieved high accuracy and generated an easily understandable decision tree.